MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

SELETIVA AULA 7

Teoria dos Números

luma palao walfrido

rev. 1.1

CC BY NC SA

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

MDC e MMC

MaratonaCIn

GCD e LCM

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

API.cpp

```cpp
// complexidade temporal: O(log(min(a, b)))

int main() {
    int a = 24, b = 36;

    // maior número que divide a e b
    int g = gcd(a, b);

    // menor número positivo múltiplo de a e de b
    int l = lcm(a, b);

    // resultados
    // gcd(24, 36) = 12
    // lcm(24, 36) = 72

    return 0;
}
```

CodeImage

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Divisores e Fatoração Prima

MaratonaCIn

Divisores

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

Como descobrir os divisores de N?

Forma ingênua: percorrer todos os valores de 1 até n, testa se cada número divide n. Complexidade O(N)

Por que essa abordagem é ruim?

- Para n grandes (ex.: 10^9) seriam 1 bilhão de testes

Time limit exceeded on test 4
Time limit exceeded on test 16
Time limit exceeded on test 27

Divisores

Forma otimizada: os divisores aparecem em pares então basta percorrer até √n

- Se i divide n então n / i também é divisor de n

- Exemplo: n = 36 e i = 4

  Divisores: 4 e 36/4 = 9

getdivisors.cpp

```cpp
// complexidade temporal: O(sqrt(n))
// complexidade espacial: O(qntdivisores(n))

vector<int> getdivisors(int n) {
    vector<int> divisors;
    for (int i = 1; i * i ≤ n; i++) {
        if (n % i == 0) {
            divisors.push_back(i);
            if (i ≠ n/i) divisors.push_back(n / i);
        }
    }

    return divisors
}
```

CodeImage

Fatoração Prima

- Para cada i, divide n repetidamente enquanto i divide n.
- O número de divisões vira a multiplicidade do fator primo i.
- Só adiciona ao resultado fatores que realmente apareceram (cnt > 0).
- Se sobrar n > 1 ao final, esse valor é primo e é adicionado.

getprimes.cpp

```cpp
// complexidade temporal: O(sqrt(n))
// complexidade espacial: O(qnt_fatores_primos(n))

vector<pair<int, int>> getprimes(int n) {
    vector<pair<int, int>> primes;
    for (int i = 2; i * i ≤ n; i++) {
        int cnt = 0;

        while (n % i == 0) {
            n /= i;
            cnt++;
        }

        if (cnt > 0) primes.push_back({i, cnt});
    }

    if (n > 1) primes.push_back({n, 1});
    return primes;
}
```

CodeImage

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Crivo

MaratonaCIn

Crivo

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20	21	22	23	24	25	26	27	28	29
30	31	32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47	48	49
50	51	52	53	54	55	56	57	58	59
60	61	62	63	64	65	66	67	68	69
70	71	72	73	74	75	76	77	78	79
80	81	82	83	84	85	86	87	88	89
90	91	92	93	94	95	96	97	98	99

Passo a passo:

1) Liste os números de 2 até n
2) Para cada número não riscado, marque seus múltiplos
3) Repita até $\sqrt{n}$
4) Os que sobraram são primos

Logo, se A é primo então todos os múltiplos de A não são primos.

Crivo Ingênuo

MaratonaCIn · Centro de Informática UFPE · Universidade Federal de Pernambuco

O laço externo percorre todos os valores até n e, para cada i, marca múltiplos.

Vários números são marcados repetidas vezes mesmo sabendo que eles não podem ser primos.

O algoritmo faz marcações desnecessárias

crivo_básico.cpp

```cpp
const int LIM = 1e6 + 5;
int divisors[LIM];

// Complexidade Temporal: O(NlogN)
// Complexidade Espacial: O(N)

vector<int> sieve() {
    memset(divisors, 0, sizeof(divisors));

    for (int i = 2; i < LIM; i++) {
        for (int j = i; j < LIM; j += i)
            divisors[j]++;
    }

    vector<int> primes;
    for (int i = 2; i < LIM; i++) {
        if (divisors[i] == 1)
            primes.push_back(i);
    }

    return primes;
}
```

CodeImage

Série Harmônica

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

Por que o Crivo Ingênuo tem complexidade O(n log(n))?

Para cada i, o loop interno roda N/i vezes para i = 2, 3, 4..., resultado numa soma igual a: **((N/2 + N/3 + N/4 + ...) = N*(½ + ⅓ + ¼ + ...)**

Sabemos que a soma (½ + ⅓ + ¼ + ...) é da ordem de O(lg N). Logo, temos a complexidade de O(N lgN)

Prova:

https://en.wikipedia.org/wiki/Harmonic_series_(mathematics)

Crivo Otimizado

MaratonaCIn · Centro de Informática UFPE · Universidade Federal de Pernambuco

Utiliza vetor de booleanos ***"isPrime"*** que indica se o número é primo ou não, então o loop interno só roda para os números primos

Agora só iteramos sobre primos relevantes

crivo_otimizado.cpp

```cpp
const int LIM = 1e6 + 5;
bool isPrime[LIM];

// Complexidade Temporal: O(Nlog(log(N)))
// Complexidade Espacial: O(N)

vector<int> sieve() {
    memset(isPrime, 1, sizeof(isPrime));
    isPrime[0] = isPrime[1] = false;

    vector<int> primes;
    for (int i = 2; i < LIM; i++) {
        if (isPrime[i]) {
            primes.push_back(i);
            for (int j = i + i; j < LIM; j += i)
                isPrime[j] = false;
        }
    }

    return primes;
}
```

CodeImage

Soma Harmônica

Por que o Crivo Otimizado tem complexidade O(n log(log n))?

A complexidade total é proporcional à soma de n/p para todos os primos p menores ou iguais a √n. A soma interna é a soma harmônica dos inversos dos números primos

$$\text{Complexidade} \approx n \times \left(\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \cdots \right)$$

Prova:
https://cp-algorithms.com/algebra/sieve-of-eratosthenes.html#asymptotic-analysis

Crivos Especiais

MaratonaCIn · Centro de Informática UFPE · Universidade Federal de Pernambuco

Calcular a soma dos divisores de todos os números no intervalo [1, LIM]

soma dos divisores do intervalo.cpp

```cpp
//Complexidade Temporal: O(N log N)
//Complexidade Espacial: O(N)

long long sumDivisors[LIM];

void sieve() {
    for (int i = 1; i < LIM; i++) {
        for (int j = i; j < LIM; j += i) {
            sumDivisors[j] += i;
        }
    }
}
```

CodeImage

Calcular a quantidade de divisores de todos os números no intervalo [1, LIM]

quantidade dos divisores do intervalo.cpp

```cpp
//Complexidade Temporal: O(N log N)
//Complexidade Espacial: O(N)

long long numDivisors[LIM];

void sieve() {
    for (int i = 1; i < LIM; i++) {
        for (int j = i; j < LIM; j += i) {
            numDivisors[j]++;
        }
    }
}
```

CodeImage

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Aritmética Modular

Exponenciação rápida
Combinatória

Aritmética Modular

Em diversos problemas, a resposta é esperada sair módulo algum número grande, como 1e9+7 ou 998 244 353 (zap)

Output

For each test case, output a single line containing an integer — the value of the function $f(a, k)$ modulo $998\,244\,353$.

Output

For each test case, output an integer, the answer, modulo $10^9 + 7$.

Isso é necessário quando a resposta numérica passa os limites built-in das linguagens ou é custosa de representar com aritmética inteira completa

Aritmética Modular

Assim, as operações matemáticas passam a pertencer ao corpo dos inteiros módulo MOD, onde todos os inteiros são mapeados para um número do conjunto {0, 1, 2, ..., MOD-1}, dessa maneira conseguimos lidar com respostas numéricas imensas dentro dos limites da linguagem

Alguns exemplos com MOD = 2 e MOD = 5:

3 + 5 = 10 (mod 2) → (3 mod 2) + (5 mod 2) = (10 mod 2) → 1 + 1 = 0

27*9 = 3 (mod 5) → (27 mod 5) * (9 mod 5) = (3 mod 5) → 2 * 4 = 3 (mod 5)

Aritmética Modular

Propriedades:

(a + b) mod MOD = (a mod MOD) + (b mod MOD)

(a * b) mod MOD = (a mod MOD) * (b mod MOD)

(a - b) mod MOD = (a mod MOD) - (b mod MOD) (cuidado com negativos)

Aritmética Modular

```
ll mod_add(ll a, ll b) {
    return (a % MOD + b % MOD) % MOD;
}

ll mod_sub(ll a, ll b) {
    return ((a % MOD - b % MOD) + MOD) % MOD;
}

ll mod_mul(ll a, ll b) {
    return (a % MOD * b % MOD) % MOD;
}
```

Aritmética Modular

E a divisão?

nos inteiros, $a/b = a * (b^{-1})$, onde $b * (b^{-1}) = 1$

então, para realizar a divisão é preciso descobrir b^{-1} dado um b e um MOD

Aritmética Modular

Quando MOD é primo usamos o pequeno teorema de fermat

$$a^p \equiv a \pmod{P},$$

$$a^{p-1} \equiv 1 \pmod{P},$$

$$a^{p-2} \equiv a^{-1} \pmod{P}.$$

Aritmética Modular

Exponenciação:

Caso especial, já que não é garantido que $(a^b) = (a \bmod MOD)^{(b \bmod MOD)}$

$5^{10} \bmod 3 \mathrel{!}= 2^1 \bmod 3$

Como fazer então?

Fast exponentiation

Jeito ingênuo de calcular a^b:

```
res = 1
for i in range(b):
    res = mod_mul(res,a):
```

O(b) multiplicações

Fast exponentiation

Jeito esperto de calcular a^b:

$$x^{n+m} = x^n \cdot x^m$$

a = 5, b = 14:

temos que b = $2^3 + 2^2 + 2^1$ (representação binária), logo:

$$5^{14} = 5^8 \cdot 5^4 \cdot 5^2$$

Fast exponentiation

$$5^{14} = 5^{8} \cdot 5^{4} \cdot 5^{2}$$

Conseguimos calcular $a^{(2^n)}$ facilmente elevando ao quadrado repetidamente

$a, a^2, (a^2)^2, ((a^2)^2)^2$

$a, a^2, \quad a^4, \quad a^8$

como b tem log(b) bits acesos, usamos log(b) multiplicações

Fast exponentiation

```
ll fexp(ll a, ll b){
    ll ret = 1;

    ll pot = a; // a^1
    for(ll bit = 0; (1LL<<bit) <= b; bit++){ //itera pelo bit
        if(b & (1LL<<bit)){ //se b tem o bit aceso, multiplica por a^(2^bit)
            ret = mod_mul(ret, pot);
        }
        pot = mod_mul(pot,pot);
    }

    return ret;
}
```

Fast exponentiation

Também é possível implementar o fexp usando a recorrência abaixo

$$a^n = \begin{cases} 1 & \text{if } n == 0 \\ \left(a^{\frac{n}{2}}\right)^2 & \text{if } n > 0 \text{ and } n \text{ even} \\ \left(a^{\frac{n-1}{2}}\right)^2 \cdot a & \text{if } n > 0 \text{ and } n \text{ odd} \end{cases}$$

Fast exponentiation

```
ll fexp(ll a, ll b){
    ll ret = 1;

    while(b){
        if(b&1) ret = mod_mul(ret,a); // se b é impar, multiplica ret por a
        a = mod_mul(a,a); // a = a^2
        b >>= 1; // b = b/2
    }

    return ret % MOD;
}
```

Combinatória modular

Boa parte dos problemas de contagem com aritmética modular são de combinatória

Contar coisas com combinatória explode os números muito rápido

$C(50,5) > 10^{19}$ (escolher 5 pessoas de um grupo de 50 pessoas)

$2^{100} > 10^{30}$ (quantos subsets um array de tamanho 100 tem)

$52! >$ número de átomos do universo (baralhos de poker)

Combinatória modular

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Combinatória modular

Podemos preprocessar os números fatoriais antes de usar:

```
fact[0] = fact[1] = 1
for i in range(2,MAXN):
    fact[i] = mod_mul(fact[i-1], i)
    invfact[i] = mod_inv(fact[i])
```

Combinatória modular

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

```cpp
ll fact[MAXN], invFact[MAXN];
void precompute() {
    fact[0] = invFact[0] = 1;
    for (int i = 1; i < MAXN; i++) {
        fact[i] = (fact[i - 1] * i) % MOD;
        invFact[i] = inv(fact[i]);
    }
}

ll binomial(ll n, ll k) {
    if (k > n || k < 0) return 0;
    return fact[n] * invFact[k] % MOD * invFact[n - k] % MOD;
}
```

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Inclusão Exclusão

MaratonaCIn

Inc-Exc

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

Time limit: 1.00 s **Memory limit:** 512 MB

You are given k distinct prime numbers $a_1, a_2, \ldots, a_k$ and an integer n.

Your task is to calculate how many of the first n positive integers are divisible by at least one of the given prime numbers.

Input

The first input line has two integers n and k.

The second line has k prime numbers $a_1, a_2, \ldots, a_k$.

Output

Print one integer: the number integers within the interval $1, 2, \ldots, n$ that are divisible by at least one of the prime numbers.

Constraints

- $1 \leq n \leq 10^{18}$
- $1 \leq k \leq 20$
- $2 \leq a_i \leq n$

Example

Input:

```
20 2
2 5
```

Output:

```
12
```

Inc-Exc

Seja M_i = múltiplos de i.

Queremos $| \cup M_{a[i]} |$.

M_x no range [1,n] é floor(n/x)

A resposta pra {2,5} e n = 10

então é 10/2 + 10/5 = 7?

Inc-Exc

M_x no range [1,n] é floor(n/x)

A resposta pra {2,5} e n = 10
então é 10/2 + 10/5 = 7?

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Estamos contando dobrado para os múltiplos de 10. Precisamos subtrair 10/10 = 1.

Inc-Exc

$$
\begin{aligned}
\left| \bigcup_{i=1}^{n} A_i \right| &= \sum_{i=1}^{n} |A_i| \\
&- \sum_{1 \le i < j \le n} |A_i \cap A_j| \\
&+ \sum_{1 \le i < j < k \le n} |A_i \cap A_j \cap A_k| \\
&- \ldots + (-1)^{n-1} \left| \bigcap_{i=1}^{n} A_i \right|
\end{aligned}
$$

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Recorrência Linear

Exponenciação de matrizes

Combinação Linear

Resultado da soma de termos onde cada termo é um produto de um elemento por uma constante escalar.

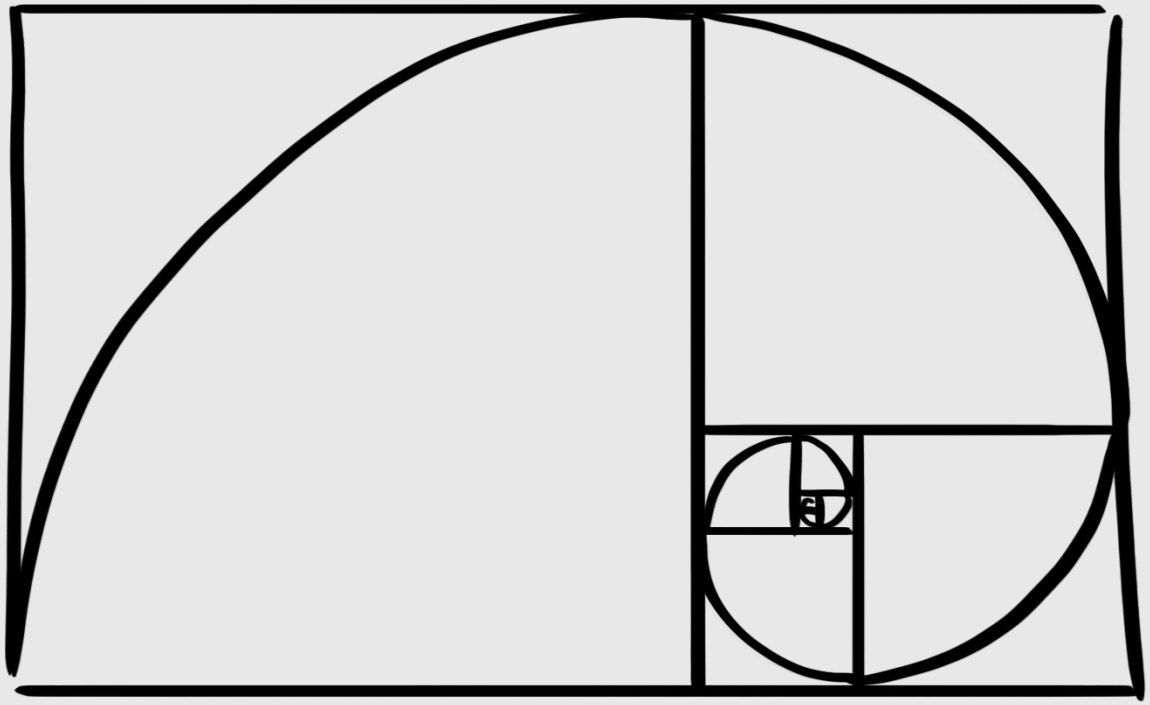
- x = 3y - 5z (x é escrito como combinação linear de y e z)

$$F_n = F_{n-1} + F_{n-2}$$

Fib(n) como combinação linear de Fib(n-1) e Fib(n-2)

Recorrência Linear

Sequência de números em que um número é descrito como combinação linear de números anteriores.



Recorrência Linear

Podemos escrever a relação de fibonacci entre termos consecutivos como multiplicação de matrizes

Recorrência Linear

Podemos escrever a relação de fibonacci entre termos consecutivos como multiplicação de matrizes

$F(n) = F(n-1) + F(n-2)$

$$\begin{pmatrix} F(n) \\ F(n-1) \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} F(n-1) \\ F(n-2 \end{pmatrix}$$

Exercícios

- $F(x) = 2.F(x-1) + 3.F(x-2)$

Exercícios

- $F(x) = 2.F(x-1) + 3.F(x-2)$
- $F(x) = 2.F(x-1) + 3.F(x-2) + 5.F(x-3) + 1$

Exercícios

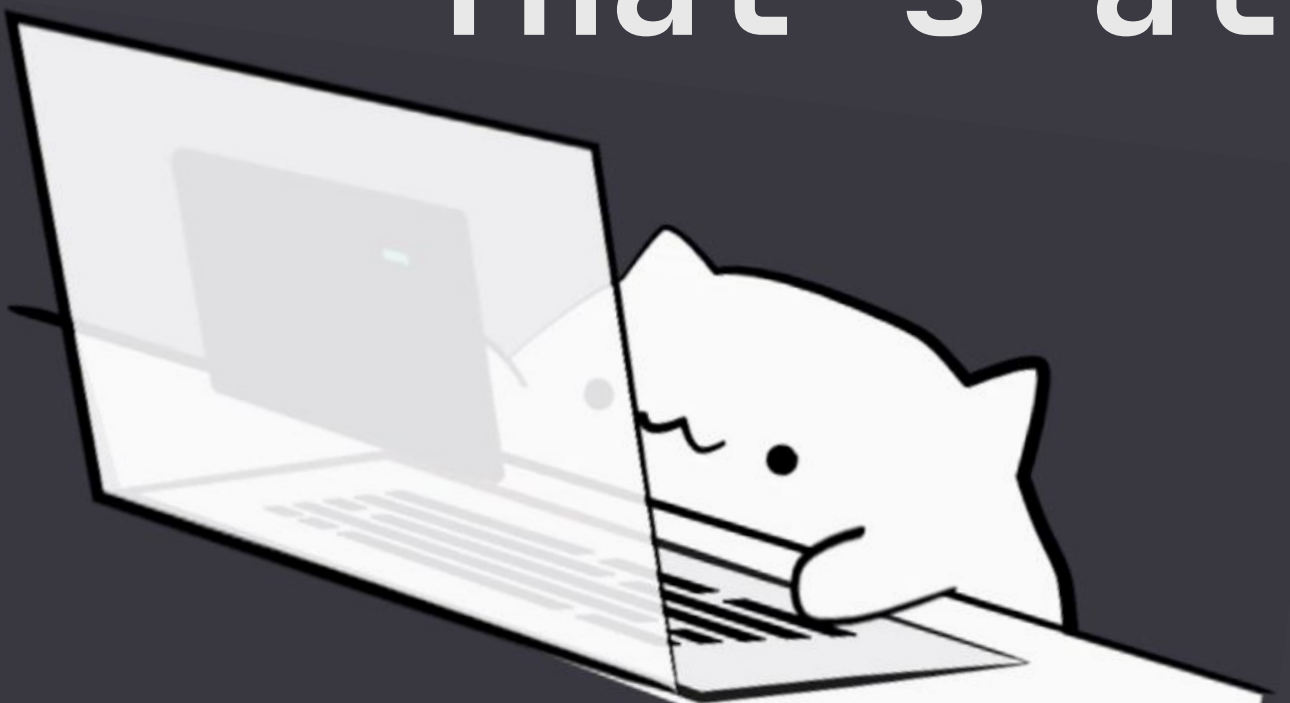
- $F(x) = 2.F(x-1) + 3.F(x-2)$
- $F(x) = 2.F(x-1) + 3.F(x-2) + 5.F(x-3) + 1$
- $F(x) = 2.F(x-1) + 3.F(x-2) + x$

Exercícios

- $F(x) = 2.F(x-1) + 3.F(x-2)$
- $F(x) = 2.F(x-1) + 3.F(x-2) + 5.F(x-3) + 1$
- $F(x) = 2.F(x-1) + 3.F(x-2) + x$
- $F(x) = 2.F(x-1) + 3.F(x-2) + x^2$
- $F(x) = 2.F(x-1) + 3.F(x-2) + 2^x$

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

That’s all, Folks!